

GPU Streaming Multiprocessor Front-End Study Guide

Nitesh Bakhati

Phase 1 – 4 · SIMT Foundations · Mini-SM Front-End · Interview Q&A;

8 Sections

18 Subsections

4 Project Phases

11+ Q&A; Items

TABLE OF CONTENTS

■ PROJECT NARRATIVE

- How to Tell This as One Coherent Story
- Resume Bullet Point Breakdown

■ SIMT FOUNDATIONS

- SIMT vs SIMD – The Critical Distinction
- Warp Semantics – What is a Warp?
- Phase 1 – SIMT Execution Substrate

■ MASKING & PREDICATION

- Phase 2 – Lane Masks and Selective Execution

■ DIVERGENCE & RECONVERGENCE

- Phase 3 – Divergence: The Core Problem
- SIMT Stack and Reconvergence Mechanics

◆ MINI-SM FRONT-END

- Phase 4 – Why the Mini-SM Front-End is the Right Next Step
- Warp Table – The Machine's View of Warp Availability
- Warp Scheduler – Arbitration and Latency Hiding
- Scoreboard – Dependency-Aware Issue Control
- Execution Pipe, Writeback & Mini-SM Top Integration

■ PERFORMANCE & MODELING

- Cycle-Accurate Modeling – What It Means and Why It Matters
- Occupancy, Latency Hiding, and Architectural Tradeoffs

■ CODE STRUCTURE & FILES

- Repository Structure & Key Files – Architectural Decomposition

■ INTERVIEW Q&A;

- Core Technical Questions with Model Answers
- Master Keywords Cheat Sheet & Industry Context

■ PROJECT NARRATIVE

■ How to Tell This as One Coherent Story

The single most powerful thing you can do in an interview is frame this project as one coherent architectural story – NOT a list of phases, modules, or code files.

The story arc is:

1. What does it mean for one instruction to drive many threads simultaneously? → SIMT execution substrate (Phase 1)
2. What if not all threads should participate in a given instruction? → Lane masking / predication (Phase 2)
3. What if threads want to take different branch paths? → Divergence & reconvergence (Phase 3)
4. How do multiple warps compete for the execution resource, hide latency, and make forward progress? → Mini-SM front-end (Phase 4)

Each phase creates the exact assumptions the next phase needs. This is not arbitrary – it mirrors how GPU architects actually reason about execution correctness before adding scheduling complexity.

Key framing distinction: Phases 1–3 address the EXECUTION MODEL (how one instruction behaves across many lanes). Phase 4 addresses the FRONT-END RESOURCE MANAGEMENT MODEL (how multiple warps compete, wait, and issue over time).

Why this matters in an interview: saying "I added a scheduler" sounds like module assembly. Saying "once execution semantics were correct, the natural next architectural question was how to choose among warp contexts under latency and dependency constraints" sounds like an architect.

■ **KEY TERMS:** SIMT · warp · execution model · front-end · throughput model · resource management · latency hiding · architectural story

■ Resume Bullet Point Breakdown

Every bullet on your resume maps to a specific architectural concept. Here's what each one means and what you must be able to defend:

BULLET 1: "Designed a pre-silicon functional model of a GPU SM front-end modeling concurrent warp execution, latency-aware instruction issue, and dependency-constrained scheduling."

- Pre-silicon functional model = C++/SystemVerilog model that captures architectural behavior before RTL is taped out
- Concurrent warp execution = multiple warps in-flight simultaneously, each with independent context
- Latency-aware instruction issue = the scheduler knows operations take multiple cycles and plans around that
- Dependency-constrained scheduling = the scoreboard prevents issuing an instruction whose source operand isn't ready

BULLET 2: "Architected warp arbitration and scoreboard-based tracking to enforce correct execution ordering under multi-cycle latency."

- Warp arbitration = selecting which eligible warp issues next (round-robin, priority, etc.)
- Scoreboard-based tracking = tagging destination registers as busy until writeback completes
- Correct execution ordering = no instruction issues when its producer hasn't finished
- Multi-cycle latency = the execution pipeline takes N cycles, not 1

BULLET 3: "Evaluated microarchitectural tradeoffs in latency hiding and occupancy by varying warp count and execution depth."

- Latency hiding = covering execution stalls by switching to other ready warps
- Occupancy = how many warps are active simultaneously relative to the maximum
- Varying warp count = running the model with 2, 4, 8 warps to observe throughput changes
- Execution depth = the latency of the execution pipe; longer = more warps needed

BULLET 4: "Structured a parameterized SM framework extensible toward full pipeline, divergence handling, and multi-warp scheduling studies."

- Parameterized = warp count, lane count, latency are compile-time or runtime parameters
- Extensible = the architecture is clean enough to add fetch/decode, memory, or L1\$ later
- Full pipeline / divergence / multi-warp scheduling = future extensions that are already architecturally motivated

■ **KEY TERMS:** pre-silicon · functional model · warp arbitration · scoreboard · occupancy · latency hiding · parameterized · extensible

■ SIMT FOUNDATIONS

■ SIMT vs SIMD – The Critical Distinction

This is one of the most important conceptual distinctions you must be able to articulate clearly. Interviewers at GPU companies will absolutely probe this.

SIMD (Classic Vector):

- A vector instruction is applied uniformly across all elements
- All lanes are assumed active at all times
- No notion of per-element control flow or conditions
- The programmer thinks in vectors: "apply this op to 8 elements simultaneously"

SIMT (GPU):

- Threads are grouped into warps; a warp shares one instruction stream
- BUT each thread retains its own logical identity, its own registers, its own condition history
- The hardware applies one instruction across all threads in a warp IN PARALLEL
- However, some threads may be logically inactive (due to conditions, branches, etc.)
- The programmer thinks in individual threads: "I write a scalar program; the GPU runs it in parallel"

The core difference: In SIMT, uniformity exists only at the instruction-stream level. Underneath, there is thread-level identity and per-thread conditions. This is why the hardware needs:

- Lane masks (to represent which threads are currently active)
- Branch control logic (to handle threads taking different paths)
- Reconvergence (to reunite threads after a divergent branch)

Key interview answer: "SIMT differs from SIMD in that threads retain individual logical identity and control flow. The hardware must represent and manage per-thread participation through masking, and must handle cases where threads within a warp take different code paths through divergence and reconvergence mechanisms."

■ KEY TERMS: SIMT · SIMD · warp · lane · lockstep · thread identity · control flow · per-thread

■ Warp Semantics – What is a Warp?

A warp is the fundamental unit of execution scheduling in a GPU.

Definition: A warp is a group of threads (typically 32 in NVIDIA hardware) that share a single program counter and execute the same instruction in lockstep each cycle.

Key warp properties:

1. SHARED PROGRAM COUNTER: All threads in a warp advance through the instruction stream together. There is one PC for the warp, not one per thread.
2. LOCKSTEP EXECUTION: On each cycle, one instruction is fetched/issued for the entire warp. All active lanes execute that instruction simultaneously.
3. INDEPENDENT REGISTER STATE: Each thread has its own logical register file. Even though they execute the same instruction, they do so on different data.

4. WARP ID: The hardware tracks warps by ID. The warp table stores per-warp state.

Why 32? (NVIDIA) – Because 32 provides a good balance:

- Large enough to amortize instruction fetch/decode overhead
- Small enough that divergence doesn't waste too many cycles
- Maps well to memory transaction granularity (coalescing)

Warp states you must know:

- READY: The warp has an instruction ready to issue and no unresolved dependencies
- STALLED: The warp's next instruction depends on an in-flight result (scoreboard blocking)
- WAITING: The warp is blocked on a long-latency operation (e.g., memory load)
- INACTIVE/DONE: No more instructions

Interview connection: "A warp is the granularity at which the front-end operates. The scheduler issues one warp's instruction per cycle. The warp table is the machine's explicit representation of which warps can be considered for issue."

■ **KEY TERMS:** warp · program counter · lockstep · 32 threads · warp state · READY · STALLED · warp table · register file

■ Phase 1 – SIMT Execution Substrate

Phase 1 answers: "If a warp executes one instruction across many threads, what does the underlying datapath look like?"

Architecture of the SIMT execution substrate:

The key design choice is SEPARATION OF CONCERNS:

- Control path: Centralized. One operation-select signal is broadcast to all lanes. This is the "one instruction" part of SIMT.
- Data path: Replicated. Each lane has its own operand inputs and produces its own result. This is the "multiple threads" part.

This separation is not an implementation detail – it is the fundamental execution geometry of a GPU.

The Lane ALU Slice:

- The elemental compute unit
- Takes: (a) shared operation select, (b) lane-local operand A, (c) lane-local operand B
- Produces: lane-local result
- Architecturally interesting NOT because of its arithmetic, but because it is the atom that gets REPLICATED to create SIMT parallelism

The SIMT ALU Core:

- Instantiates N lane ALU slices (N = warp width, e.g. 8 in your model)
- Broadcasts the op_sel to all slices
- Fans in N pairs of operands, fans out N results
- This fan-out IS warp-level parallel execution

Parameterization philosophy:

Your design uses parameters for lane count. This signals you are not building a fixed toy. A parameterized lane array can be scaled, which mirrors how GPU architects think about PE arrays and thread block sizing.

Verification mindset for SIMT:

In scalar design, you test one output. In SIMT, a valid test must verify ALL lanes simultaneously under common instruction control. The test question is: "For this operation and these N sets of lane operands, did every lane produce the correct result?"

Key interview phrasing: "Phase 1 established warp-level parallel execution by mapping one instruction across N lane-local datapaths under common control. The critical design choice was separating the centralized instruction interpretation from the replicated per-lane data path."

■ **KEY TERMS:** lane ALU · op_sel · fan-out · replicated datapath · common control · parameterized · SIMT
ALU core · lane slice · synthesizable

■ MASKING & PREDICATION

■ Phase 2 – Lane Masks and Selective Execution

Phase 2 answers: "What if not all threads in a warp should participate in a given instruction?"

The core concept – Active Lane Mask:

A lane mask (also called active mask or predicate mask) is an N-bit vector (N = warp width) where each bit represents whether the corresponding thread lane is currently architecturally active.

Bit = 1 → Thread is active, its result is architecturally valid and should update state

Bit = 0 → Thread is inactive, its result should NOT update visible architectural state

This is "selective architectural effect" – the key phrase for interviews.

What masking enables:

1. CONDITIONAL EXECUTION: Thread 3 failed an if-condition → bit 3 = 0, result ignored
2. LOOP REMAINDER HANDLING: Threads beyond array bounds → masked out
3. BRANCH SERIALIZATION SETUP: Divergent warps execute one path with a mask, then the other path with the complementary mask
4. INACTIVE THREAD SAFETY: State of inactive threads is preserved, not corrupted

Inactive lane correctness – why it matters:

Inactive lanes are NOT "don't care" lanes. They are lanes whose architectural state MUST be preserved unchanged. A common class of real hardware bugs happens when:

- An inactive lane's result leaks into a register update
- A condition check accidentally affects an inactive lane's PC state
- A write-back doesn't properly gate on the active mask

Bridge to Phase 3 – why masking is a prerequisite for divergence:

A branch split is fundamentally a change in which subset of lanes is active for a given execution path. Without a formal notion of active-lane subsets, divergence has no vocabulary to express itself. Phase 2 creates that vocabulary. Phase 3 uses it.

Key interview phrasing: "Phase 2 introduced selective architectural effect – the mechanism by which the execution block respects per-thread participation without altering the arithmetic datapath. A lane mask is a compact representation of which threads are logically active for a given instruction, and correctness requires that inactive lanes not corrupt their own architectural state."

■ **KEY TERMS:** active mask · lane mask · predicate mask · selective architectural effect · inactive lane correctness · architectural state · composability · N-bit vector

■ DIVERGENCE & RECONVERGENCE

■ Phase 3 – Divergence: The Core Problem

Phase 3 is where the project becomes unmistakably GPU-specific. This is the most conceptually rich phase.

The Divergence Problem:

A warp has N threads. They share one program counter. They reach a conditional branch: `if (threadIdx.x < 4)`. In an 8-wide warp: threads 0-3 evaluate TRUE, threads 4-7 evaluate FALSE.

What happens? The warp CANNOT follow both paths simultaneously with all threads. But it also CANNOT simply pick one path and abandon the other – that would violate program correctness for the threads on the ignored path.

The solution: PATH SERIALIZATION

Execute the TRUE path with mask = `0b00001111` (threads 0-3 active), THEN execute the FALSE path with mask = `0b11110000` (threads 4-7 active). Both paths are executed, but sequentially, not in parallel.

Architectural consequence: DIVERGENCE REDUCES EFFECTIVE UTILIZATION

While executing the TRUE path, threads 4-7 are idle (SIMD efficiency = $4/8 = 50\%$). While executing the FALSE path, threads 0-3 are idle (efficiency = 50% again). This is the fundamental performance cost of control-flow divergence in GPU hardware.

Key insight for interviews: Divergence doesn't create two independent warps in the programming model. One original warp is temporarily partitioned into active subsets. The warp must eventually RECONVERGE – return to unified execution.

Why the hardware needs state:

The machine must REMEMBER:

- Which subset of lanes still has work to do on the deferred path
- Where that deferred path should resume (PC value)
- Under what active mask the deferred path should execute
- The correct ORDER in which paths should be executed and reconverged

This memory is the SIMT Stack.

■ **KEY TERMS:** divergence · path serialization · active mask split · SIMD efficiency · utilization · deferred path · control-flow divergence · branch

■ SIMT Stack and Reconvergence Mechanics

The SIMT Stack – What it actually is:

The SIMT stack preserves deferred execution context across branch splits. It is not merely "storage" – it is the state machine that enforces correct ordering of diverged execution paths and ensures reconvergence.

Each stack entry contains:

- Active mask: which threads are valid for this context
- PC / next PC: where this context should execute
- Reconvergence PC (RPC): the PC at which this diverged subset should eventually join back

Stack operation during a branch:

1. Warp reaches branch instruction with current mask `M_current`
2. Branch evaluates: `M_true` = threads that went true, `M_false` = threads that went false
3. Push entry for FALSE path: (`mask=M_false`, `PC=false_target`, `RPC=reconverge_point`)
4. Push entry for TRUE path: (`mask=M_true`, `PC=true_target`, `RPC=reconverge_point`)
5. Execute TRUE path until reconverge point
6. Pop TRUE entry, execute FALSE path until reconverge point
7. Pop FALSE entry, restore full mask → all threads unified

Nested divergence:

The stack architecture handles nested branches naturally. If a branch occurs INSIDE an already-reduced-mask region, new entries are pushed on top. The stack correctly tracks the order in which sub-contexts must be resolved before the outer context can reconverge. This is exactly why a stack (LIFO) structure is the right abstraction.

Reconvergence – the precise definition:

Reconvergence is NOT "threads meeting again." It is the restoration of correct unified control after temporary path serialization. It discharges the deferred execution obligations stored in the stack and returns the warp to a coherent state.

Interview depth question: "What happens in nested divergence?"

→ "The SIMT stack handles this by pushing contexts in order. Inner divergence pushes more entries on top. Each reconvergence pops one context. The LIFO ordering ensures inner branches reconverge before outer branches, preserving correct program semantics."

■ **KEY TERMS:** SIMT stack · divergence stack · reconvergence PC · RPC · nested divergence · stack entry · active mask split · deferred context · LIFO · path serialization

◆ MINI-SM FRONT-END

■ Phase 4 – Why the Mini-SM Front-End is the Right Next Step

After Phase 3, you have a correct SIMT execution engine. Phase 4 wraps that engine in front-end control machinery that turns it into a throughput-oriented system.

The conceptual transition:

Before Phase 4: SIMT execution engine (correctness-focused)

→ One warp, one instruction, behavioral correctness

After Phase 4: SM front-end (throughput-focused)

→ Multiple warp contexts, competing for execution resources

→ Some ready, some stalled, some blocked on dependencies

→ Scheduler picks eligible warps to maximize utilization

→ Scoreboard enforces dependency correctness

→ Execution pipe introduces real latency → latency hiding becomes observable

Why this step and not something else?

You could have added a cache, a fetch unit, or memory subsystem. But the Mini-SM front-end is the **STRONGEST** next step because:

→ The execution logic is mature enough to support it

→ It introduces the most architecturally rich concepts: scheduling, scoreboarding, latency hiding, occupancy

→ It completes the "SM" narrative without requiring a full pipeline

→ These are exactly the concepts GPU architect and modeling interviews test

The issue-complete loop (the heartbeat of the Mini-SM):

1. Warp table: which warps are ready?
2. Scheduler: pick an eligible warp
3. Scoreboard check: are all source operands ready?
4. Issue: send instruction to execution pipe
5. Execution pipe: N-cycle delay
6. Writeback: result returns, scoreboard clears, warp potentially becomes re-eligible
7. Go to step 1

This loop, repeated every cycle, IS the SM front-end.

■ **KEY TERMS:** issue-complete loop · throughput model · front-end · SM · warp context · eligible warp · writeback · occupancy

■ Warp Table – The Machine's View of Warp Availability

What is the warp table?

The warp table is the hardware structure that stores per-warp state, specifically whether each warp is currently eligible to be issued. It is the scheduler's window into which warps can be considered this cycle.

Per-warp state it tracks:

→ **READY:** Warp has a valid instruction, all source operands available, active mask valid

- STALLED: Warp's next instruction depends on an in-flight result (scoreboard blocking it)
- WAITING: Warp is pending a long-latency event (memory, sync barrier)
- DONE/INACTIVE: No more instructions to issue

Why separate warp state from the scheduler?

This is an important architectural decomposition principle. The warp table owns STATE BOOKKEEPING. The scheduler owns SELECTION POLICY. These are independent concerns:

- You can change the scheduler policy without changing how state is stored
- You can change what constitutes "ready" without changing the selection algorithm

State transitions:

READY → STALLED: Instruction issued, scoreboard marks destination busy, next instruction needs that register

STALLED → READY: Writeback completes, scoreboard clears busy, warp's dependency resolved

READY → WAITING: Instruction issued is a long-latency op (e.g., memory load)

WAITING → READY: Long-latency op completes, result available

Interview connection: "The warp table gives the scheduler a precise, cycle-accurate view of which warps can be issued. It separates the bookkeeping of warp readiness from the selection policy, which is important for modularity and for studying different scheduling algorithms independently."

■ **KEY TERMS:** warp table · per-warp state · READY · STALLED · WAITING · state transition · bookkeeping · selection policy · warp_id

■ Warp Scheduler – Arbitration and Latency Hiding

What does the scheduler actually do?

The warp scheduler examines the warp table each cycle, identifies eligible (READY) warps, and selects one to issue. In your Mini-SM, this is round-robin: cycle through warp IDs, find the next READY one, issue it.

Why round-robin is architecturally meaningful:

- It is DETERMINISTIC – easy to debug, reason about, and compare against expected behavior
- It is FAIR – no warp is systematically starved
- It already demonstrates the key throughput insight: if Warp 0 is stalled, Warp 1 can issue
- Real GPU schedulers (GTO, Two-Level, etc.) are variants on this baseline

Latency Hiding – the central throughput concept:

Without multiple warps: When Warp 0 issues an instruction with N-cycle latency, the machine is IDLE for N-1 cycles. This is pure performance waste.

With multiple warps + scheduler: While Warp 0 waits for its result (N cycles), the scheduler can issue from Warp 1, Warp 2, ... Warp K. If $K \geq N$, the machine can potentially issue every cycle despite individual warps stalling.

The math: To fully hide L cycles of execution latency, you need at least L independent ready warps.

Occupancy:

$OCCUPANCY = (\text{active warps}) / (\text{maximum possible warps for this SM})$

Higher occupancy → more candidates for the scheduler → better latency hiding → higher throughput

Your experiment: "Evaluated microarchitectural tradeoffs in latency hiding and occupancy by varying warp count and execution depth."

- Low warp count + high latency: many idle cycles, throughput suffers
- High warp count + matching latency: near-100% issue utilization

Scheduler policies to know:

- Round-robin (RR): Cycle through warps in order. Fair, simple, your baseline.
- Greedy-then-oldest (GTO): Stick with same warp until it stalls, then pick oldest ready.
- Two-level scheduler: Prioritize a fetch group, round-robin within the group.
- Loose-round-robin (LRR): NVIDIA baseline – round-robin with some priority adjustments.

■ **KEY TERMS:** round-robin · GTO · latency hiding · occupancy · throughput · issue slot · idle cycle · eligible warp · arbitration · fairness · stall

■ Scoreboard – Dependency-Aware Issue Control

What is the scoreboard?

The scoreboard tracks which architectural registers have in-flight (unfinished) write operations. When an instruction is issued and will produce a result, its destination is MARKED BUSY. Any subsequent instruction that wants to READ that destination is BLOCKED from issuing until writeback clears the busy bit.

Scoreboard mechanics:

ISSUE: Instruction issued for Warp W, producing result in Reg R

- Scoreboard entry for (Warp W, Reg R) = BUSY

NEXT INSTRUCTION CHECK: Warp W's next instruction needs Reg R

- Scoreboard says (W, R) = BUSY → BLOCK ISSUE for Warp W

WRITEBACK: Execution pipe completes after L cycles, writes result for (W, R)

- Scoreboard clears (W, R) → Warp W may now be re-eligible

Why per-warp? Because register namespaces are per-warp. Warp 0's R2 is completely independent of Warp 1's R2. So the scoreboard is indexed by (warp_id, reg_id).

Simple scoreboard vs. Tomasulo:

Simple scoreboard (your project): Tracks busy registers per warp. No out-of-order within a warp.

Tomasulo / reservation stations: Full out-of-order execution, register renaming, eliminates WAR and WAW hazards. Much more complex. Not needed for in-order warp scheduling.

Hazard types you must know:

RAW (Read After Write): True data dependency – instruction needs a value not yet produced. THE critical hazard. Scoreboard directly prevents this.

WAR (Write After Read): Anti-dependency – would overwrite a value before it's been consumed.

WAW (Write After Write): Output dependency – two writes to same register in wrong order.

Interview phrasing: "The scoreboard mediates the relationship between correctness and throughput. It converts execution latency – which is a timing property – into explicit issue constraints. Without it, a warp could issue an instruction whose source operand hasn't been written back yet, producing incorrect results."

■ **KEY TERMS:** scoreboard · busy bit · RAW hazard · WAR · WAW · writeback · dependency · in-flight · per-warp register · issue constraint · Tomasulo

■ Execution Pipe, Writeback & Mini-SM Top Integration

Execution Pipe – Why it matters architecturally:

Without latency, the scoreboard and scheduler have no real work to do. Every instruction would complete instantly, every warp would always be ready, and the front-end would be a trivial counter. Latency is what makes the front-end non-trivial.

The execution pipe introduces a fixed delay of L cycles between issue and writeback – a shift-register-style pipeline. When an instruction is issued (`warp_id`, `reg_id`, `operands`), it enters the pipe. After L cycles, it exits as a writeback event (`warp_id`, `reg_id`, `result`).

Writeback is the event that discharges dependency obligations:

This causal chain – issue → pipe → writeback → scoreboard clear → warp re-eligible – is the central rhythm of the Mini-SM.

Cycle-by-cycle mental model (use this in interviews):

Cycle 1: Scheduler picks Warp 0. Scoreboard marks R2 busy (latency = 4). Warp 0 → STALLED.

Cycle 2: Warp 0 stalled. Scheduler picks Warp 1. Issues instruction.

Cycle 3: Warp 0 still stalled. Scheduler picks Warp 2.

Cycle 5: Warp 0's instruction writes back. Scoreboard clears R2. Warp 0 → READY.

Cycle 6: Scheduler considers Warp 0 again. Issues its next instruction.

Mini-SM Top Integration:

The `mini_sm_top` wires together: warp table + scheduler + scoreboard + execution pipe + SIMT execution block into one coherent issue-complete loop.

Issue side: Scheduler → (`warp_id`, instruction) → Scoreboard (mark busy) + Execution pipe (start computation) + Warp table (update state)

Completion side: Execution pipe → (`warp_id`, `reg_id`, `result`) → Scoreboard (clear busy) → Warp table (potentially re-enable warp)

Without top integration, you have a collection of modules. With it, you have a system that proves multiple warp contexts can coexist, compete, make progress, and maintain correctness simultaneously.

■ **KEY TERMS:** execution pipe · writeback · shift register pipeline · fixed latency · cycle-accurate · `mini_sm_top` · issue-complete loop · integration · interface contract

■ PERFORMANCE & MODELING

■ Cycle-Accurate Modeling – What It Means and Why It Matters

Functional model vs Cycle-accurate model:

Functional model: Captures WHAT the architecture does. Correct input → correct output. No timing information. Can't tell you how long something takes or when events happen.

Cycle-accurate model: Captures WHAT and WHEN. Every state transition is tied to a specific clock cycle. You can ask "how many cycles did this take?" and get a meaningful answer.

Your Mini-SM is cycle-accurate because:

- The execution pipe has an explicit latency in cycles
- The scoreboard tracks when (which cycle) a result will be available
- The scheduler makes decisions based on the CURRENT cycle's state
- Issue events, stall events, and writeback events all have cycle timestamps

What "pre-silicon" means:

"Pre-silicon" = before the chip is fabricated. Pre-silicon models serve multiple purposes:

- Architecture exploration: Compare design A vs design B before committing to RTL
- Software bringup: Develop firmware/drivers before hardware exists
- Performance prediction: Estimate throughput, identify bottlenecks
- Verification planning: Understand expected behavior for test development

Metrics your model can produce:

- Issue rate: instructions issued per cycle (ideally 1.0 if never stalled)
- Stall rate: cycles where no warp was issued due to all being blocked
- Utilization: fraction of cycles with a valid issue
- Warp-level IPC: instructions per cycle per warp
- Latency coverage: fraction of execution latency covered by other warps

■ **KEY TERMS:** cycle-accurate · functional model · pre-silicon · throughput · issue rate · stall rate · utilization · IPC · performance prediction · architecture exploration

■ Occupancy, Latency Hiding, and Architectural Tradeoffs

Occupancy – precise definition:

Occupancy = (active warps on SM) / (max warps SM can hold)

Higher occupancy → more warp contexts → scheduler has more candidates → better latency hiding → higher throughput.

But occupancy is not everything:

- If all warps are stalled on the same long-latency event, high occupancy doesn't help
- Occupancy is a NECESSARY but not SUFFICIENT condition for high throughput
- Real GPUs have resource limits: registers per thread, shared memory per block, thread block count limits

The arithmetic of latency hiding:

If execution latency = L cycles, you need AT LEAST L warps to fully hide latency. With fewer warps, throughput ≈ (warp_count / L) × peak_throughput.

Architectural Tradeoffs – "What if" questions:

1. What if you increase execution latency?
 - Scoreboard entries stay busy longer → need MORE warps → if fixed warp count, throughput decreases
2. What if you increase warp count?
 - More candidates for scheduler → better latency hiding → BUT register file grows linearly
 - Beyond the hiding point, additional warps provide diminishing returns
3. What if all warps are dependent? (Heavy dependency)
 - Scoreboard blocks all warps simultaneously → scheduler has zero eligible warps
 - Issue rate drops → this is "dependency-limited" regime, not occupancy-limited
4. What if you replace round-robin with greedy?
 - Better cache locality, but: if current warp stalls quickly, still frequent context-switches
 - Tradeoff: locality vs. fairness vs. stall sensitivity
5. Why need both warp table AND scoreboard?
 - Warp table: coarse-grained warp readiness (ready/stalled/waiting)
 - Scoreboard: fine-grained register-level dependency (which specific reg is busy)
 - A warp can be READY in warp table but blocked by scoreboard for a specific instruction
6. What is the area cost of increasing lane count?
 - Each additional lane adds one ALU slice (roughly linear area)
 - Register file grows: $N_{\text{warps}} \times N_{\text{regs}} \times N_{\text{lanes}} \times \text{reg_width}$ bits
 - Interconnect complexity grows (result buses need to be wider)

■ **KEY TERMS:** occupancy · latency hiding · throughput · warp count · compute-bound · latency-bound · dependency-limited · roofline · area cost · lane count

■ CODE STRUCTURE & FILES

■ Repository Structure & Key Files – Architectural Decomposition

Repository structure reflects the block decomposition of the machine:

rtl/pkg/ – Shared type and parameter definitions

- simt_alu_pkg.sv: Central definition of WARP_COUNT, LANE_COUNT, REG_WIDTH, EXEC_LATENCY, warp state enums, instruction structs
- No magic numbers scattered across files. Changing warp count in one place propagates everywhere.

rtl/alu/ – Lane-level arithmetic (Phases 1-2)

- lane_alu.sv → simt_alu_core.sv → simt_alu_top.sv

rtl/control/ – Branch and divergence control (Phase 3)

- branch_ctrl.sv, simt_stack.sv: Divergence management, active mask splitting, SIMT stack, reconvergence

rtl/core/ – Mini-SM front-end blocks (Phase 4)

- warp_table.sv, warp_scheduler.sv, scoreboard.sv, exec_pipe.sv

rtl/top/ – Integration

- mini_sm_top.sv: Wires all blocks, instantiates the issue-complete protocol end-to-end

File-level explanations you must give fluently:

lane_alu.sv: "The repeatable execution atom. Its parameterization allows N copies to form the warp-width SIMT datapath."

simt_alu_core.sv: "This is where one instruction becomes N-parallel computations – the execution geometry of a warp."

warp_table.sv: "This separates state bookkeeping from selection policy – the scheduler reads it but doesn't own it."

scoreboard.sv: "This converts execution latency into explicit issue constraints, turning the scheduler from a warp picker into a dependency-aware issue engine."

warp_scheduler.sv: "The policy engine that converts available warp contexts into productive issue slots."

exec_pipe.sv: "Without this, scheduling has no cost model. This is what makes the model cycle-accurate rather than merely functional."

mini_sm_top.sv: "This makes the project an SM front-end rather than a collection of independent modules. The integration proves the issue-complete loop works end-to-end."

■ **KEY TERMS:** rtl/pkg · rtl/alu · rtl/control · rtl/core · rtl/top · simt_alu_pkg · lane_alu · simt_alu_core · warp_table · scoreboard · warp_scheduler · exec_pipe · mini_sm_top

■ INTERVIEW Q&A;

■ Core Technical Questions with Model Answers

Q: Walk me through your GPU SM project.

A: "I built a pre-silicon functional model of a GPU SM front-end in four architectural phases. I started with the execution substrate – a lane-parallel SIMT datapath where one instruction drives N thread lanes simultaneously under common control. I then added selective lane participation through active-mask predication, which lets some threads be inactive while preserving their architectural state. After that, I modeled divergence and reconvergence – the mechanism by which a warp survives branch splits through path serialization and a SIMT stack. Finally, I wrapped this execution machinery in a Mini-SM front-end: a warp table for state tracking, a round-robin scheduler for arbitration, a scoreboard for dependency-constrained issue, and an execution pipe that makes latency real. The result is a cycle-accurate model that demonstrates latency hiding and throughput behavior across multi-warp configurations."

Q: What is the difference between SIMT and SIMD?

A: "In classical SIMD, all vector elements participate uniformly in every operation – there's no notion of per-element control flow. In SIMT, threads retain individual logical identity and can have divergent conditions. The hardware uses lane masking to represent which threads are architecturally active for a given instruction, and a divergence stack to serialize execution of different code paths within the same warp."

Q: How does your scoreboard work?

A: "The scoreboard is a per-(warp, register) busy bit array. When an instruction is issued that will produce a result in register R for Warp W, the scoreboard marks (W, R) as busy. If the next instruction from Warp W needs R, the scoreboard blocks its issue. When the execution pipe completes after L cycles and writeback occurs, the busy bit clears. This converts execution latency into explicit issue constraints."

Q: What is latency hiding and how does your model demonstrate it?

A: "Latency hiding is the technique of keeping the execution unit busy during the stall period of any individual warp by issuing from other ready warps. When Warp 0 issues an instruction with L-cycle latency, the scoreboard marks its destination busy. For the next L cycles, the scheduler can issue from other ready warps instead of idling. I demonstrated this by sweeping warp count from 2 to 8 with different execution depths and observing throughput changes."

Q: What happens during branch divergence in your model?

A: "When a warp reaches a branch, some threads evaluate true and others false. The branch control logic determines the two active-mask subsets. A stack-based structure stores the deferred path. The machine executes the first path under its mask until the reconvergence point, pops the stack, and executes the second path. At reconvergence, the full mask is restored. The cost is path serialization – during each path, the other half of lanes are idle."

Q: How does your warp table differ from your scoreboard?

A: "The warp table tracks coarse-grained warp readiness – whether a warp as a whole is ready, stalled, or waiting. The scoreboard tracks fine-grained register-level dependency. A warp might be READY in the warp table but blocked by the scoreboard if its next instruction reads an in-flight result. Together, they give the scheduler complete visibility."

Q: Why do you need a SIMT stack for reconvergence rather than just a register?

A: "A single register would only track one deferred context. Nested divergence creates multiple layered obligations. The SIMT stack handles this naturally – each branch pushes a new entry, inner branches reconverge first. The LIFO ordering ensures inner divergence is resolved before outer, which matches the program's control flow semantics."

Q: What limits your Mini-SM's throughput in the best case?

A: "In the ideal case – enough warps to fully hide latency, no dependencies blocking all warps, and fair round-robin scheduling – throughput is bounded by 1 instruction issued per cycle. 1 IPC is the ceiling. Real GPUs have additional constraints: fetch bandwidth, decode throughput, operand read port limits – not modeled in the Mini-SM front-end."

Q: How would you extend this to model memory latency?

A: "Memory operations have much longer latency (hundreds of cycles for DRAM, tens for L2). I would add a memory-latency model that assigns a much larger latency value to load/store instructions. The scoreboard would hold the busy bit for many more cycles, showing that you need far more warps to hide memory latency than arithmetic latency."

Q: Why did you model divergence BEFORE adding the scheduler?

A: "Divergence and reconvergence are properties of execution semantics – they define what it means for a warp to correctly traverse control flow. The scheduler is front-end resource management. A scheduler is only meaningful if the underlying execution semantics are already correct."

Q: What would happen if your scheduler was unfair (always picked the same warp)?

A: "Other warps would never issue – they'd starve. This would reduce effective multi-warp utilization to zero. In a real GPU, scheduler unfairness could cause some thread blocks to finish much later than others, causing load imbalance."

■ **KEY TERMS:** interview answer · walk me through · SIMT vs SIMD · scoreboard explanation · latency hiding · divergence · warp table vs scoreboard · nested divergence · throughput ceiling · memory latency

■ Master Keywords Cheat Sheet & Industry Context

EXECUTION MODEL TERMS:

- Warp: Group of threads (typically 32) sharing one PC, executing in lockstep
- Lane: One thread's execution slot within a warp's parallel datapath
- SIMT: Single Instruction, Multiple Threads – GPU's execution model
- Lane mask / active mask / predicate mask: N-bit vector encoding thread participation per instruction
- Selective architectural effect: Inactive lanes don't update visible state
- Divergence: Threads within a warp taking different branch paths

- Path serialization: Executing divergent paths sequentially under alternating masks
- Reconvergence: Restoring unified warp execution after serialized divergence
- Reconvergence PC (RPC): The instruction address where diverged threads reunite
- SIMT stack: Hardware structure preserving deferred execution context during divergence

FRONT-END / SCHEDULING TERMS:

- Warp table: Per-warp state register array (READY/STALLED/WAITING)
- Warp scheduler: Arbitration logic selecting which eligible warp issues next
- Scoreboard: Per-(warp, reg) busy-bit structure preventing RAW hazard violations
- RAW hazard: Read After Write – true data dependency requiring a wait
- Writeback: Completion event that resolves scoreboard busy entries
- Execution pipe: Fixed-latency model between issue and writeback
- Issue slot: One opportunity per cycle to issue an instruction
- Latency hiding: Using other warps' instructions to fill issue slots during a warp's stall
- Occupancy: Ratio of active warps to maximum possible warps on the SM
- Throughput: Issue rate – ideally 1 instruction per cycle

MODELING TERMS:

- Pre-silicon: Before chip fabrication – model exists instead of silicon
- Functional model: Captures correct behavior, no timing
- Cycle-accurate model: Captures correct behavior WITH cycle-level timing
- Parameterized: Design scales with compile-time or runtime parameters
- Synthesizable: RTL that can be mapped to real logic gates or FPGA LUTs

PERFORMANCE TERMS:

- IPC (Instructions Per Cycle): Throughput metric – higher = more efficient
- Stall cycle: A cycle where no warp issued due to all being blocked
- Utilization: Fraction of cycles with a valid issue event
- Dependency-limited: Throughput limited by internal data dependencies within warps
- Occupancy-limited: Throughput limited by insufficient active warps to hide latency

INDUSTRY / REAL HARDWARE TERMS:

- GTO (Greedy-Then-Oldest): NVIDIA's default scheduler policy in many generations
- Two-level scheduler (TLS): Separates warp selection into two stages for better cache behavior
- SIMD group: Apple GPU term for warps (32 threads)
- TBDR: Tile-Based Deferred Rendering – Apple GPU rendering architecture
- Register file banking: Partitioning the register file to allow parallel accesses
- Bank conflict: Multiple warps accessing the same memory bank simultaneously
- Warp-level primitives: CUDA intrinsics exposing warp-level operations (__shfl, __ballot)

APPLE GPU / NVIDIA CONNECTION:

- Apple "SIMD groups" = NVIDIA warps, both 32 threads
- Apple focuses on efficiency: instructions per watt, not just raw throughput
- NVIDIA Ampere/Hopper: 2 warp schedulers per SM, each issuing one instruction/cycle
- Your CUDA roofline project: Each CUDA thread = one lane; each warp = one warp table entry
- Your 2.7x tiling improvement = moving from memory-bound to compute-bound regime on roofline

■ **KEY TERMS:** warp · lane · SIMT · active mask · divergence · reconvergence · RPC · SIMT stack · scoreboard · RAW · writeback · occupancy · IPC · pre-silicon · cycle-accurate · Apple GPU · NVIDIA SM · GTO · two-level scheduler · SIMD group · TBDR